

altairsim — Recipe: a Dazzler graphics machine with a Z80 in it

2026-09-23 · dbdb362

A machine with no operating system, no disk, and a color display: a Cromemco Dazzler, a Z80, and 64K of RAM, running Li-Chen Wang's Kaleidoscope. You will build it from an empty chassis, watch it draw, save it, quit, and load it back.

It is also the answer to a question the shipped machines never ask out loud: **how do I put a Z80 in this thing?** A processor is a board like any other board. It is one line.

Same four beats as every recipe here:

```
altairsim -n — an empty backplane. Fit the boards — BOARDS ADD , REGION ADD , SET , STARTUP  
ADD . CONFIG SAVE — write the machine down. Quit, altairsim -n again, CONFIG LOAD — and it  
comes back.
```

Work in a copy of `examples/dazzler/`, which holds the program this machine runs.

```
$ cd examples/dazzler
```

1. Fit a Z80

```
$ altairsim -n  
altairsim> BOARDS ADD z80 cpu0 clock_hz=4000000  
cpu0: z80 added
```

That is the whole of it. `z80` instead of `8080` on the `BOARDS ADD` line, and the machine has a Z80 in it — the id stays `cpu0`, and nothing else in the machine changes, because nothing else knows or cares which processor is driving the bus. `SHOW BOARDS` lists the other processors you could have fitted instead.

`clock_hz=4000000` is 4 MHz, the speed a Cromemco Z-1 ran. `0` means flat out and is the default; a period pace is what makes the animation look like the period.

Everything the machine does from here is the same as it would be with an 8080 — `SHOW`, `SET`, `RUN`, the debugger. What changes is that `DISASM` speaks Z80, and that Z80 programs run.

2. The rest of the machine

A terminal to type at, the Dazzler, and memory.

```
altairsim> BOARDS ADD 2sio sio0 port=10
sio0: 2sio added
altairsim> SET sio0:a connect=console
sio0:a: connect=console
altairsim> BOARDS ADD dazzler daz0 port=0x0E
daz0: dazzler added
altairsim> BOARDS ADD memory mem0 fill=random
mem0: memory added
altairsim> REGION ADD mem0 type=ram at=0 size=64K
mem0:0: ram 0000-FFFF 64K
```

SET takes one property per command — `SET sio0:a connect=console baud=9600` is refused with `SET: unexpected 'baud=9600'`. And a board has to exist before you can `SET` on it, so the `BOARDS ADD` always comes first.

Fitting a memory board gives you no memory. The board is the card; what is on it is a list of regions, and `REGION ADD` puts one there. This machine is 64K of RAM and no ROM at all — there is no boot PROM, because there is nothing to boot. The program is loaded by hand.

The Dazzler has no memory of its own. Its picture is ordinary RAM (KSCOPE puts it at `0200`), and the two ports at `0E`/`0F` are all the card decodes. `fill=random` is how static RAM really powered up, and it is why the picture emerges out of color noise.

3. The window

```
altairsim> SET DISPLAY focus=on
display: focus=on
altairsim> SET DISPLAY keyboard=none
display: keyboard=none
```

`focus=on` brings the video window to the front when it opens, instead of leaving it behind your terminal. `keyboard=none` says the window is a display and not a keyboard — which is what a Dazzler is — so you keep typing at the terminal. Ctrl-E in either one still stops the guest.

These are properties of the host's video window, not of any board, which is why they are `SET DISPLAY` and not `SET daz0`.

4. Load the program and start it

```
altairsim> STARTUP ADD LOAD KSCOPE.HEX
added: LOAD KSCOPE.HEX
altairsim> STARTUP ADD RUN 0
added: RUN 0
altairsim> SET MACHINE name=kscope-z80
machine: name=kscope-z80
```

STARTUP ADD records a command to run whenever this machine loads — here, read KSCOPE into memory and start the processor at **0000**. You can type both by hand instead; the startup list is just how a machine file remembers the keystrokes.

SET MACHINE name= names the machine. A machine built from **-n** is called **none** until you say otherwise, and the name is what **CONFIG SAVE** writes into the file.

Before saving, look at the whole thing. **SHOW MACHINE** is the machine you have built — its name, its startup list, and every board in the backplane:

```
altairsim> SHOW MACHINE
name      kscope-z80
startup
          LOAD KSCOPE.HEX
          RUN 0

  ID  TYPE  I/O  UNITS      MEMORY
  ---  -
cpu0  z80   -    1 cpu: z80   -
sio0  2sio  10,12  2 serial: a*, b -
daz0  dazzler 0E,0F -           -
mem0  memory -    -           0000-FFFF ram 64K

* holds the console
```

Four boards, and one of them is a Z80. (**BOARDS** prints the table alone, without the name and the startup list; **SHOW <id>** prints one board in full.)

5. Save it, quit, load it back

```
altairsim> CONFIG SAVE kscope-z80.toml
saved kscope-z80.toml
altairsim> QUIT
```

```
$ altairsim -n
altairsim> CONFIG LOAD kscope-z80.toml
loaded kscope-z80.toml: 4 board(s)
startup> LOAD KSCOPE.HEX
loaded 127 bytes (1 page) from KSCOPE.HEX (0000-007E)
startup> RUN 0
[console -- ^E returns to the monitor]
```

On a build with SDL3 the window opens here and the kaleidoscope draws: a 2 KB, 64×64, 16-color picture, mirrored four ways. KSCOPE never stops on its own, so press Ctrl-E to break back to the `altairsim>` prompt — `RUN 0` starts it again. On a headless build the machine runs exactly the same and draws nothing.

`$ altairsim kscope-z80.toml` does the same as the two lines above.

The file opens with the name you gave it and the two startup lines you recorded:

```
[machine]
name      = "kscope-z80"
startup   = [
  "LOAD KSCOPE.HEX",
  "RUN 0",
]
```

See that it really is a Z80

Break out with Ctrl-E and disassemble the first few instructions:

```
altairsim> DISASM 0 6
0000 31 00 01 LD SP,0100
0003 3E 81    LD A,81
0005 D3 0E    OUT (0E),A
0007 3E 30    LD A,30
0009 D3 0F    OUT (0F),A
000B 78      LD A,B
```

`LD A,81` and `OUT (0E),A` are Z80 mnemonics; the same bytes on an 8080 board would have printed `MVI A,81` and `OUT 0E`. The bytes did not change — the processor reading them did. (KSCOPE is 8080 code, and a Z80 runs it unchanged, which is the whole point of the Z80.)

What to do next

Compare your file with `examples/dazzler/kscope.toml` : the same machine, written by hand, with the reasoning in comments. And `recipes/cpm-from-scratch.pdf` builds the other kind of machine — disk, boot PROM, and an operating system.